

Desafio Técnico -- Analista Sênior (C#/.NET) | Instituição Financeira

Contexto

Você foi contratado para desenvolver uma API que gerencie **contratos de financiamento de veículos** em uma instituição financeira. O sistema deve permitir cadastrar contratos, registrar pagamentos e gerar um resumo consolidado do cliente.

Além do CRUD básico, o desafio foca em **regra de negócio, qualidade de código, segurança e boas práticas de arquitetura**.

Escopo

Funcionalidades obrigatórias

1. Contratos

- CRUD de contratos de financiamento (`POST` , `GET` , `DELETE`).
- Campos obrigatórios:
 - `id` (GUID)
 - `clienteCpfCnpj`
 - `valorTotal`
 - `taxaMensal` (%)
 - `prazoMeses`
 - `dataVencimentoPrimeiraParcela`
 - `tipoVeiculo` (`AUTOMOVEL` , `MOTO` , `CAMINHAO` , etc.)
 - `condicaoVeiculo` (`NOVO` , `USADO` , `SEMINOVO`)

2. Pagamentos

- Registrar pagamento de parcelas (`POST`). (`POST /api/contratos/{id}/pagamentos`).
- Listar pagamentos por contrato.
- Regras mínimas:
 - Calcular saldo devedor atualizado.
 - Identificar se o pagamento foi **em dia**, **em atraso** ou **antecipado**.

3. Resumo do Cliente (`GET /api/clientes/{cpfCnpj}/resumo`)

- Quantidade de contratos ativos.
 - Total de parcelas.
 - Parcelas pagas, em atraso e a vencer.
 - Percentual de parcelas pagas em dia.
 - Saldo devedor consolidado.
-

Requisitos técnicos

- API em **C#** / **ASP.NET Core 8+**.
- Banco de dados: relacional (**PostgreSQL**) ou NoSQL (**MongoDB**)

- Documentação de endpoints com **Swagger/OpenAPI**.
 - **Testes automatizados** para regras de negócio.
 - Projeto containerizado com **Docker + docker-compose**.
 - Logs estruturados (mínimo: request/response + erros) com Correlation ID.
-

Extras (avaliados positivamente, mas opcionais)

- Validação de **CPF/CNPJ**.
 - Paginação e ordenação na listagem de contratos.
 - Autenticação via **JWT** (mínimo: rota protegida por token fixo).
 - Uso de **DTOs** e separação clara de camadas (ex.: Controllers, Services, Repositórios).
 - README com instruções claras.
-

Entregáveis

- Repositório público (GitHub/GitLab).
 - Código-fonte completo com instruções no README.
 - `docker-compose.yml` para execução local.
 - Scripts de inicialização do banco (se necessários).
 - Testes unitários e/ou de integração.
-

Critérios de Avaliação

- Clareza e organização do código.
- Modelagem adequada de contratos e pagamentos.
- Correção das regras de negócio (saldo, atrasos, antecipação).
- Uso de boas práticas (camadas, DTOs, SOLID, Clean Code).
- Qualidade dos testes entregues.
- Clareza do README e facilidade de execução.
- **Desenvolvimento e commits incrementais.**

Pesos

- Arquitetura & Design (10%)
- Modelagem de Domínio & Dados (20%)
- Regras de Negócio (25%)
- Qualidade de Código & Manutenibilidade (10%)
- Testes Automatizados (10%)
- API & DX (Swagger/OpenAPI) (7%)
- Observabilidade & Logging (5%)
- Segurança (5%)
- Docker & Setup (5%)
- Documentação (3%)